Programmieren 1
Studienleistung - Aufgabe 1

Wintersemester 2026/27
Prof. Dr. J. Rexilius Abgabetermin: -

*Beachten Sie die allgemeinen Abgabehinweise.*

*Erstellen Sie für jede Aufgabe eine neue Klasse, die nur von `Agent` abgeleitet ist, damit die einzelnen Aufgaben separat funktionieren.*

## Teil 1 (0 Punkte)

**Szenario**

Sie sind als Astronaut auf einem unbekannten Planeten gestrandet. Nach einer Notlandung befinden Sie sich in einem weitläufigen Forschungsgebiet. Ihre Funkverbindung zur Raumstation ist unterbrochen und wichtige Versorgungspakete sind über das Gebiet verteilt.

Das Gebiet besteht aus begehbaren und nicht begehbaren Bereichen. Es kann verschiedene Wege, Abzweigungen und Sackgassen geben. Der Astronaut muss daher seine Umgebung beobachten und selbstständig entscheiden, in welche Richtung er sich bewegt. Zusätzlich kann er unterschiedliche Dinge auf seinem Weg einsammeln oder ablegen.

Die technische Umsetzung des Spiels übernimmt die bereitgestellte Engine. Sie kümmert sich um das Spielfeld, die Darstellung des Astronauten, Items und den Spielablauf. **Ihre Aufgabe besteht ausschließlich darin, das Verhalten des Astronauten in der Klasse Player zu programmieren.**

### (a) Aufgabe 1 – Das unbekannte Gebiet erkunden

Für diese Aufgabe erhalten Sie eine vorbereitete Klasse `Player`. Die Klasse ist bereits so eingerichtet, dass sie von `Agent` erbt. Die Bedeutung von Klassen und Vererbung ist für diese Aufgabe noch nicht erforderlich.

Ihre Aufgabe besteht darin, das Verhalten des Astronauten in der Methode `act()` zu programmieren. Die Methode `act()` wird von der Spiel-Engine bei jedem Tic (Takt/Zeiteinheit im Spiel) einmal aufgerufen. Bei jedem Aufruf soll Ihr Astronaut anhand seiner aktuellen Umgebung entscheiden, welche Aktion er als Nächstes ausführt. Verwenden Sie dazu nur die von `Agent` bereitgestellten Methoden zur Bewegung und Wahrnehmung der Umgebung.

Das Spielfeld ist dem Astronauten nicht vollständig bekannt. Er kann nur die Informationen verwenden, die ihm seine Sensoren zur Verfügung stellen. Entwickeln Sie eine Strategie, mit der der Astronaut das unbekannte Gebiet erkundet und das Ziel erreicht. Wenn der Astronaut das Ziel erreicht hat, soll er stehen bleiben – also stoppen. Das Gebiet kann verschiedene Wege, Abzweigungen und Sackgassen enthalten. Ihr Astronaut muss daher auf seine aktuelle Umgebung reagieren und darf nicht davon ausgehen, dass ein bestimmter Weg zum Ziel führt. Die Bewegungslogik darf nicht aus einer fest vorgegebenen Folge von Befehlen bestehen. Eine Lösung wie

```
move();
move();
turnLeft();
move();
```

ist daher nicht zulässig, da sie einen konkreten Weg voraussetzt. Stattdessen soll Ihr Player bei jedem Aufruf von `act()` die Umgebung untersuchen und daraus seine nächste Aktion ableiten. Die Position des Ziels und der konkrete Verlauf des Weges dürfen nicht im Programmcode hinterlegt werden. Ihre Lösung muss auf unterschiedlichen Spielfeldern funktionieren.

Wintersemester 2026/27
Prof. Dr. J. Rexilius Abgabetermin: -

### (b) Aufgabe 2 – Missionen erfüllen

In dieser Aufgabe soll der Astronaut während seiner Erkundung Versorgungspakete erkennen und automatisch aufnehmen. Er soll speichern, wie viele Pakete (Items vom Typ `PACKAGE`) er bereits gesammelt hat. Je nach Level müssen auch noch andere Items aufgenommen werden.

Nachdem der Astronaut die benötigte Anzahl an Items gesammelt hat, soll er die Erkundung beenden, sich zum Ziel begeben und dort stoppen. Auf dem Weg zum Ziel sollen keine weiteren Pakete oder andere Items mehr gesucht und aufgenommen werden. Der Astronaut darf das Ziel nicht als erreicht betrachten, nur weil er irgendwann auf der entsprechenden Kachel steht.

Auf dem Spielfeld kann es zusätzlich noch einzelne Gefahrenstellen geben (Kacheln vom Typ `HAZARD`). Diese muss der Astronaut entschärfen bevor er am Ziel stoppen kann. Dafür muss er ein Item vom Typ `SAMPLE` auf die Gefahrenstelle legen.

Erstellen Sie eine neue Klasse `Player` mit der entsprechenden Funktionalität. Die Position der Items darf nicht im Programmcode hinterlegt werden. Ihr Player muss anhand der tatsächlichen Spielsituation entscheiden, wann ein Paket aufgenommen werden kann. Die API der Klasse Agent stellt Ihnen dafür die entsprechenden Methoden zur Verfügung.

### (c) Aufgabe 3 – Notsignal

Um ein Notfallsignal an die Raumstation zu senden, muss der Astronaut Marker aufsammeln, diese zu einem Muster auf dem Spielfeld ablegen und dann zum Ziel gehen um dort zu stoppen.

Erstellen Sie eine neue Klasse `Player`, die von `Agent` erbt. Der Astronaut soll selbstständig die folgende Mission durchführen: (1) Alle Marker (Item-Typ `MARKER`) auf dem Spielfeld aufsammeln. (2) Danach auf die Startposition für das Notfallsignal (Item-Typ `COIN`) gehen. (3) Ausgehend von dieser Position soll der Astronaut aus den eingesammelten Markern ein quadratisches Muster auf dem begehbaren Bereich des Spielfeldes erstellen. (4) Der Astronaut soll am Ende zur Zielposition gehen und dort stoppen.

**Wichtig:**

- Anders als die anderen Bewegungen bzw. Missionsphasen soll das vollständige Zeichnen des Musters **innerhalb eines einzigen Aufrufs von `act()`** erfolgen. Das Muster soll nicht über mehrere Spiel-Tics hinweg aufgebaut werden.
- Die Größe des Quadrats soll sich aus der Anzahl der eingesammelten Marker ergeben. Das Quadrat darf nicht fest auf eine bestimmte Größe programmiert werden.
- Sie können für Ihre Implementierung annehmen, dass sich (1) links von der Startposition eine Wand befindet, und (2) nach oben und unten ausreichend Platz für das Muster ist.

## Teil 2 (10 Punkte)

In diesem Teil werden Aufgaben gestellt, die zu den Themengebieten der Aufgaben aus Teil 1 gehören. Grundlage der Aufgaben ist Ihr in Teil 1 entwickelter Quellcode. Sie erhalten dabei z.B. Aufgaben, bei denen Sie Ihre Klasse `Player` gezielt erweitern, verändern oder korrigieren müssen.

Die Aufgaben beziehen sich auf typische Änderungen, die auch bei der Weiterentwicklung eines Spiels auftreten können. Beispielsweise kann eine bestehende Schleife angepasst, der Wertebereich einer Variablen verändert, eine neue Methode ergänzt oder eine zusätzliche Klasse erstellt werden.

Wintersemester 2026/27
Prof. Dr. J. Rexilius Abgabetermin: -

**Allgemeine Hinweise und Richtlinien**

- Wird die Deadline nicht eingehalten, so wird die Aufgabe als nicht abgegeben gewertet.
- Jeder Studierende bearbeitet die Aufgaben selber und lädt die eigenen Lösungen hoch.

**Hinweise und Richtlinien für die Abgabe von Quellcode**

- Verwenden Sie folgende Java-Version: ab Java SE 25
- Geben Sie als Quellcode nur die .java-Dateien ab.
- Nutzen Sie keine JDK Enhancement Proposals (JEP).
- Code, der nicht kompiliert, wird mit null Punkten bewertet.
- Ihre Lösungen dürfen nicht zu Programmabstürzen führen.
- Dokumentieren Sie Ihren Quellcode.
- Die Schritte zur Lösung müssen eindeutig nachvollziehbar sein.
- Verwenden Sie aussagekräftige Namen für Klassen, Methoden, Variablen, etc.
    - Klassennamen beginnen groß
    - Variablennamen beginnen klein

**Neubewertung**

Wenn Sie der Meinung sind, dass es bei der Bewertung Fehler gibt, können Sie eine Neubewertung per E-Mail fordern. Diese muss maximal eine Woche nach Bekanntgabe der Bewertungen in ILIAS beim Dozenten eingehen. Beschreiben Sie darin, warum die Bewertung der abgegebenen Lösung nicht korrekt ist.

Hinweis:
Die abgegebene Lösung kann noch einmal komplett neu bewertet werden. Dies kann zu neuen Punktabzügen an anderen Stellen führen, die vorher übersehen wurden.